MODELOS BAYESIANOS

Probabilidade aplicada à Indústria 4.0

Prof. André Felipe Savedra Cruz

Contexto Industrial

No chão de fábrica, milhares de peças metálicas são usinadas por hora.

Precisamos de um sistema que:

- Avalie as dimensões (Diâmetro) e massa (Peso) da peça em **tempo real**.
- Classifique a peça como **Aprovada** ou **Refugada** (com defeito).
- Seja rápido o suficiente para não atrasar a esteira de produção.



O que são Modelos Bayesianos?



Estatística

Diferente do k-NN que mede "distância geométrica" entre os dados, os algoritmos bayesianos são puramente baseados em **Probabilidade**.



Teorema de Bayes

Eles utilizam o Teorema de Bayes, que atualiza a probabilidade de uma hipótese à medida que novas evidências ficam disponíveis na medição.



Exemplo Prático

"Qual a probabilidade desta peça ser refugada, **DADO QUE** seu peso analisado instantaneamente é de 295g?"

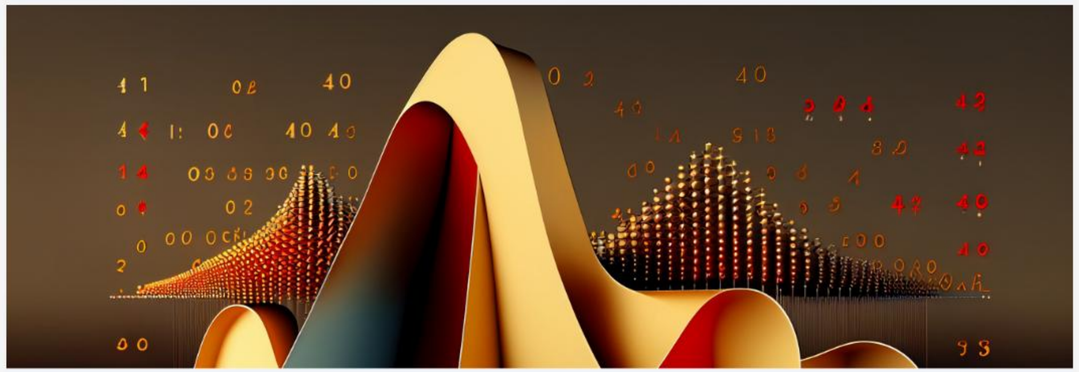
Naive Bayes (Bayes "Ingênuo")

O modelo é chamado de "Ingênuo" porque faz uma suposição muito forte:

- Assume que todas as características (ex: Peso e Diâmetro) são **completamente independentes** umas das outras.

Embora isso raramente seja verdade na engenharia (peso e diâmetro são interligados física e matematicamente), o algoritmo funciona de maneira excelente na prática industrial.

Para dados numéricos contínuos (sensores de chão de fábrica), usamos a variação **Gaussian Naive Bayes**.



Prof. André Felipe Savedra Cruz

Naive Bayes vs. k-NN



Custo Computacional

O k-NN salva todo o dataset na memória. O Naive Bayes apenas salva a Média e o Desvio Padrão. É extremamente mais rápido e leve.



Tempo de Predição

Em uma esteira veloz, o Bayes classifica os produtos em milissegundos, pois realiza apenas um cálculo de equação de probabilidade.



Escalonamento

Não sofre com features em escalas diferentes (ex: Gramas vs. ºC), dispensando o uso obrigatório do `StandardScaler` exigido pelo k-NN.

Prof. André Felipe Savedra Cruz

Passo 1: Importando Ferramentas

Na biblioteca de machine learning **scikit-learn**, o algoritmo matemático que utilizaremos está empacotado dentro do módulo especializado `naive_bayes`.

Vamos preparar o ambiente importando as ferramentas de análise e o modelo Gaussiano.

```
# Ferramentas clássicas de manipulação e gráficos
import pandas as pd
import seaborn as sns

# Importando o Algoritmo Bayesiano Gaussiano
from sklearn.naive_bayes import GaussianNB
```


Prof. André Felipe Savedra Cruz

Passo 2: Análise das Distribuições

O modelo assume que os dados têm um formato de "Sino de Gauss" (Distribuição Normal).

Plotamos curvas de densidade (KDE - *Kernel Density Estimation*) para verificar a probabilidade antes do treinamento.

```
# Plotando a distribuição das características
sns.kdeplot(data=df_pecas, x="Diametro_mm", hue="Status")

# O eixo X mostrará o diâmetro, o eixo Y mostrará
# a concentração probabilística das amostras.
```


Prof. André Felipe Savedra Cruz

Passo 3: Treinando o Algoritmo

No momento em que executamos o método `.fit()`, a Inteligência Artificial entra em ação.

Ela extrai e memoriza apenas a **Média** e a **Variância Matemática** de cada coluna em relação aos rótulos alvo.

```
# Instanciamos o modelo
modelo_bayes = GaussianNB()

# Treinamento: A matemática entra em ação
modelo_bayes.fit(X_train, y_train)

print("Modelo treinado - Equações geradas!")
```

Passo 4: Predição e Avaliação

O modelo já extraiu as probabilidades a partir do treinamento.

Agora, comparamos o que o modelo previu matematicamente com as respostas reais que separamos previamente na fase de teste.

```
# Previsão nas peças não vistas
y_pred = modelo_bayes.predict(X_test)

# Acurácia
acc = accuracy_score(y_test, y_pred)
print(f"Acurácia no lote de teste: {acc *
100:.2f}%")
```

Resumo Operacional

- **Fundação Estatística:** Modelos Bayesianos não calculam distâncias, são baseados puramente em estatística e inferência probabilística.
- **Aplicações Contínuas:** O algoritmo GaussianNB é excelente e adequado para dados numéricos contínuos provenientes de processos industriais.
- **Alta Performance:** Sua extrema velocidade o torna preferível sobre o k-NN em processamentos de Big Data ou arquiteturas IoT Industrial operando em Tempo Real.

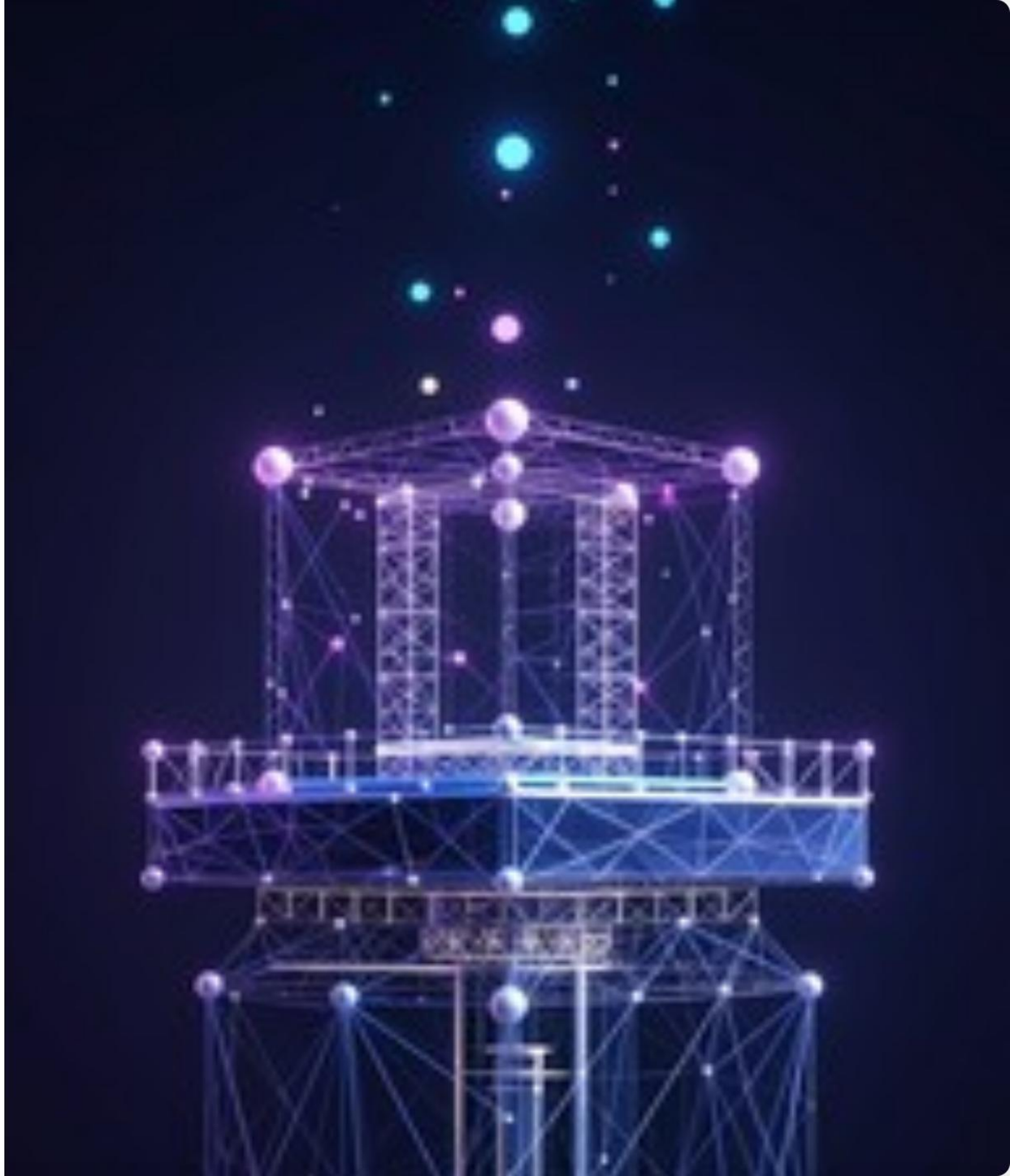
Nota de Transição: Agora, vamos aplicar esse conhecimento no exercício prático!

Exercício Prático: Smart Grid IoT

Cenário: Você atua na engenharia de uma subestação de energia inteligente (Smart Grid). A rede recebe leituras de centenas de microssensores espalhados pela cidade relatando a Tensão (Volts) e a Corrente (Amperes) em tempo real.

O sistema precisa classificar instantaneamente se o setor elétrico está **Estável (0)** ou em risco de **Queda de Fase (1)**.

Devido ao imenso volume de dados chegando por segundo, o processador não aguenta executar o k-NN. Você deve aplicar o Naive Bayes.



O Desafio no Google Colab

</> Tarefas Práticas

- Gere um dataset sintético de 5000 amostras usando `make_classification` (com 3 características contínuas).
- Treine um modelo `GaussianNB` com 75% dos dados separados para treino.
- Realize as predições na base de teste e plote a Matriz de Confusão para análise visual.

⚠ Discussão Crítica

Tensão elétrica e Corrente possuem altíssima correlação na física (Lei de Ohm). Sabendo que o Naive Bayes pressupõe rigorosamente que os sensores são independentes, justifique técnica e criticamente como essa "ingenuidade" do algoritmo afeta a performance da classificação neste contexto elétrico.

Prof. André Felipe Savedra Cruz

Image Sources



https://www.ptsmake.com/wp-content/uploads/2025/08/ptsmake2025.08.14-11185-Axis-CNC-Machining-Complex-Parts.webp

Source: www.ptsmake.com



https://mirsazzathossain.me/images/content/exploring-gaussian-distribution/cover.png

Source: mirsazzathossain.me



https://images.stockcake.com/public/a/c/5/ac51072c-eb2d-4c66-bef8-27f66b1b3f5b_medium/digital-tower-rising-stockcake.jpg

Source: stockcake.com